supporting the extra complexity of such a data structure.

### Table 9.1 Operations provided by k-d tree, and their cost on a balanced k-d tree with n elements

| Operation | Running Time | Extra Space |
|---|---|---|
| Search | O(log(n)) | O(1) |
| Insert | O(log(n)) | O(1) |
| Remove | $O(n^{1-1/k})^*$ | O(1) |
| findMin | $O(n^{1-1/k})^*$ | O(1) |
| nearestNeighbor | $O(2^k + \log(n))^*$ | $O(m)^§$ |
| pointsInRegion | O(n) | O(n) |

(*) Amortized, for a k-d tree holding k-dimensional points.

(§) When searching for the m **nearest neighbors, with m constant (not a function of** n**).**

## 9.4 Limits and Possible Improvements

If we look back at our “find the closest hub” problem, we had started this chapter with basically nothing better than brute-force search to find the nearest neighbor of a point in a multi-dimensional dataset; then, going through a few less-than-ideal attempts, we built our understanding of the traps and challenges in such a task, and finally introduced k-d trees.